



Chapter 14

Dynamic Programming

Divide and Conquer

- หลักการสำคัญ : แก้ปัญหาโดยการแก้ปัญหาย่อยๆ ซ้ำๆ ไปเรื่อยๆ แล้วนำผลลัพธ์ที่ได้นำมาใช้ในการแก้ปัญหที่ใหญ่ขึ้น
- ประเด็นปัญหา : การทำปัญหาย่อยซ้ำๆ ทำให้โปรแกรมทำงานช้า
- ทำอย่างไรให้ประมวลผลได้เร็ว -> จำค่าผลลัพธ์ที่เคยคำนวณมาแล้ว

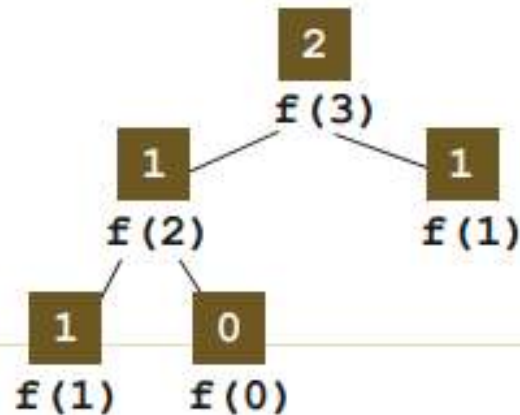
Fibonacci

$$f_n = f_{n-1} + f_{n-2}$$

$$f_0 = 0, f_1 = 1$$

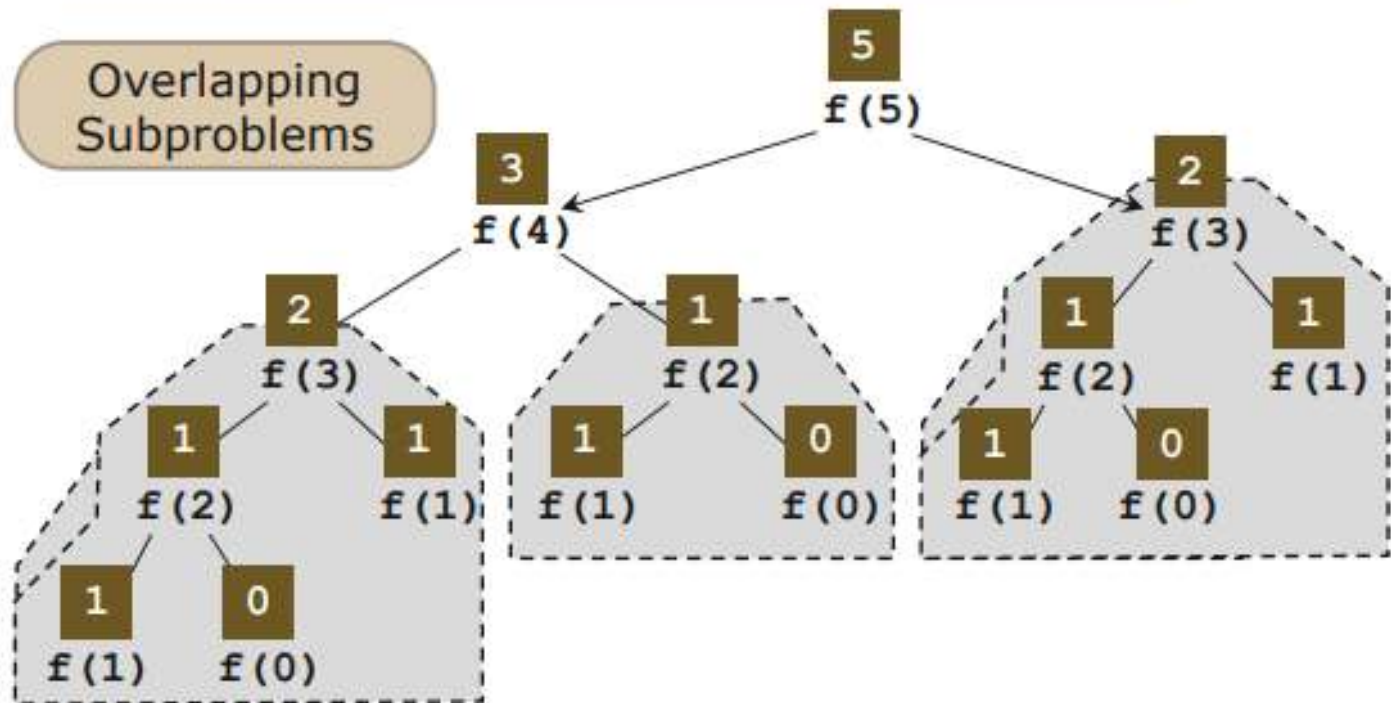
Top-down

```
f( n ) {  
    if (n < 2) return n  
    return f(n - 1) + f(n - 2)  
}
```



Fibonacci : ทำ $f(n)$ ซ้ำๆ

```
f( n ) {  
    if (n < 2) return n  
    return f(n - 1) + f(n - 2)  
}
```

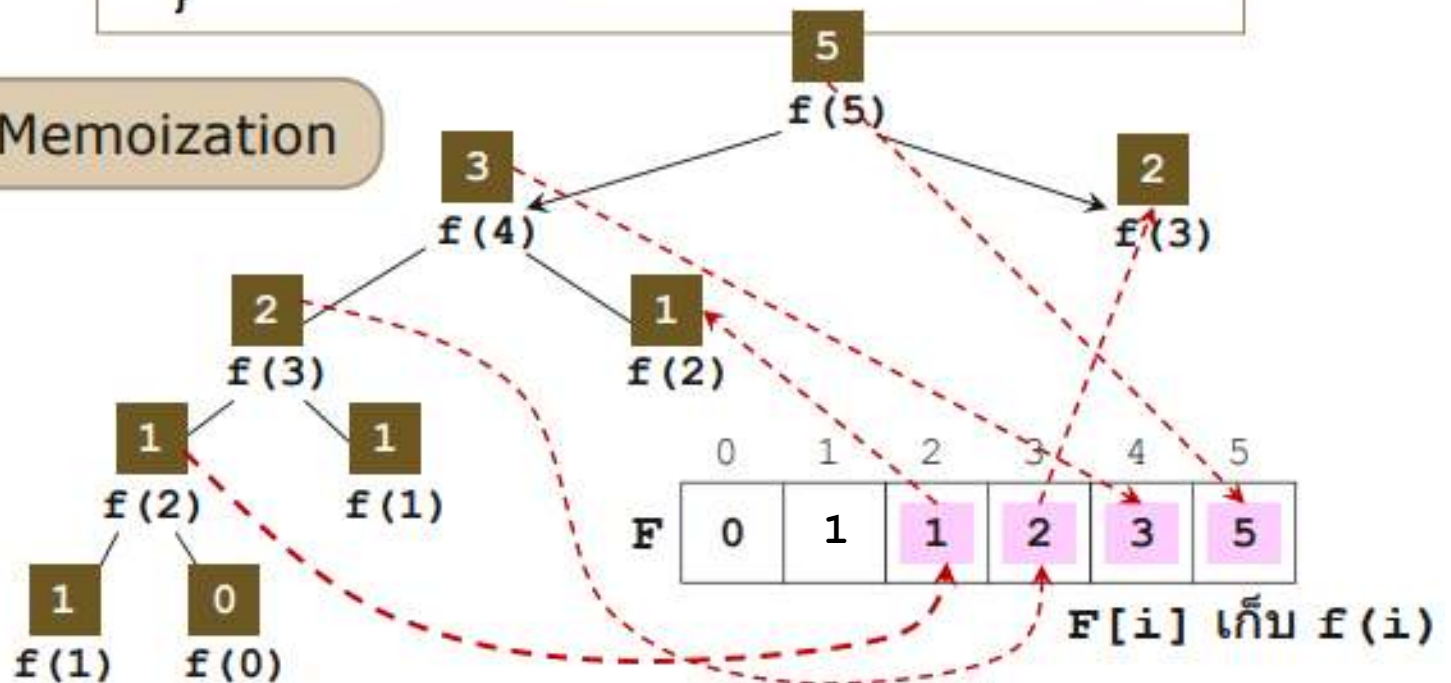


Fibonacci : จำค่า $f(n)$ ที่เคยคำนวณแล้ว

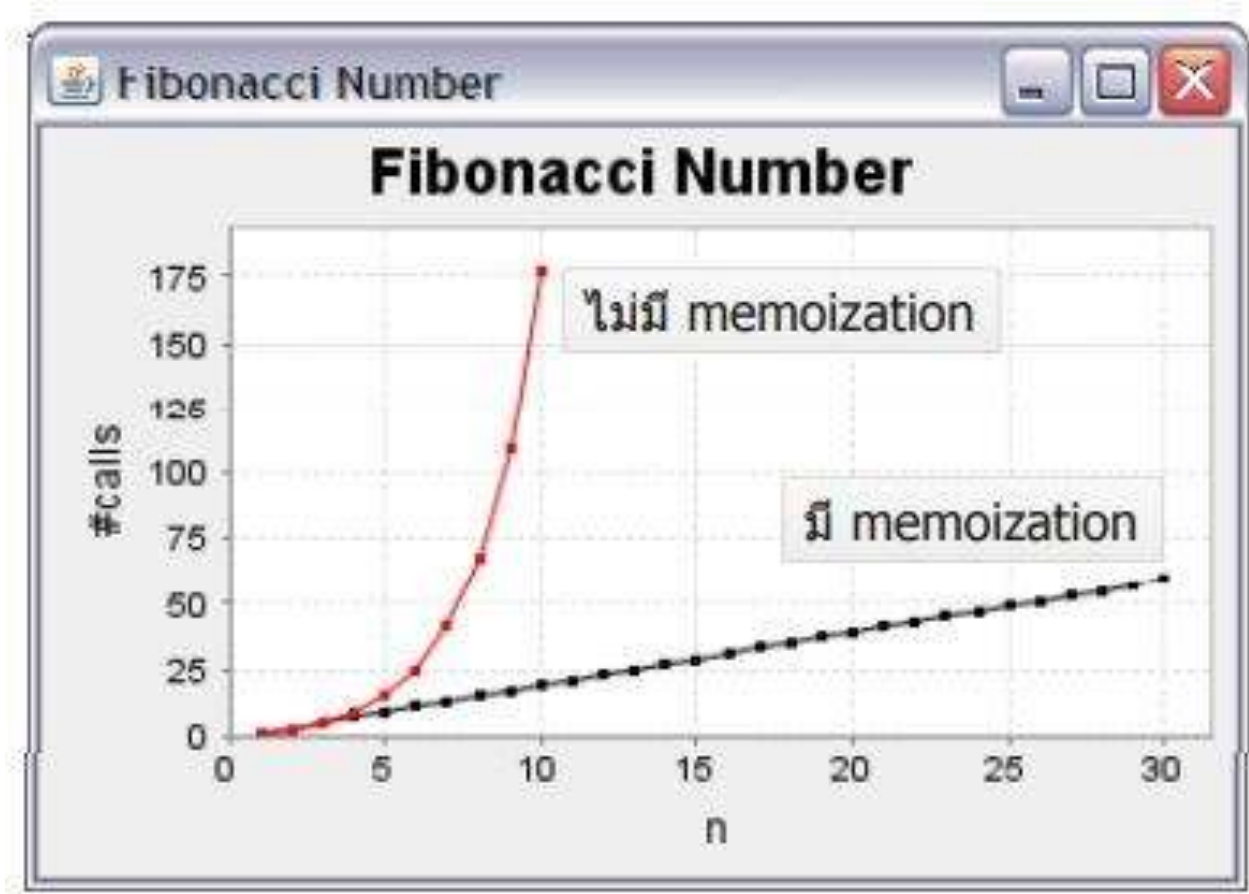
```
f(n, F[0..n]) {  
    if (n < 2) return n  
    if (F[n] > 0) return F[n]  
    return F[n] = f(n-1, F) + f(n-2, F)  
}
```

$\Theta(n)$

Memoization



Fibonacci



Fibonacci : Bottom-Up

	0	1	2	3	4	5
F	0	1	1	2	3	5

```
f( n ) {  
    F = new array[0..n]  
    F[0] = 0; F[1] = 1  
    for (i = 2; i <= n; i++) {  
        F[i] = F[i - 1] + F[i - 2]  
    }  
    return F[n]  
}
```

$\Theta(n)$

bottom-up

Dynamic Programming

- การทำงานโดยทำการแก้ปัญหาย่อย จดจำผลลัพธ์ และนำไปใช้แก้ปัญหาที่ใหญ่ขึ้นต่อไป
- มักใช้กับ Optimization problems
 - เลือกหยิบของที่มีมูลค่ารวมมากที่สุด
 - หาข้อความที่เหมือนกันมากที่สุด
 - หาเส้นทางสั้นสุด

ปัญหาที่ใช้ Dynamic Programming

- Knapsack problem
- Longest Common Substring
- Longest Common Subsequence

Knapsack problem

- ปัญหา : จะหยิบสิ่งของใส่เป้ (หรือตะกร้า) อย่างไร
 - ได้ของที่มีมูลค่ารวมมากที่สุด
 - ไม่ให้เกินน้ำหนักที่เป้จะสามารถรับได้



STEREO
\$3000
4 lbs



LAPTOP
\$2000
3 lbs



GUITAR
\$1500
1 lbs

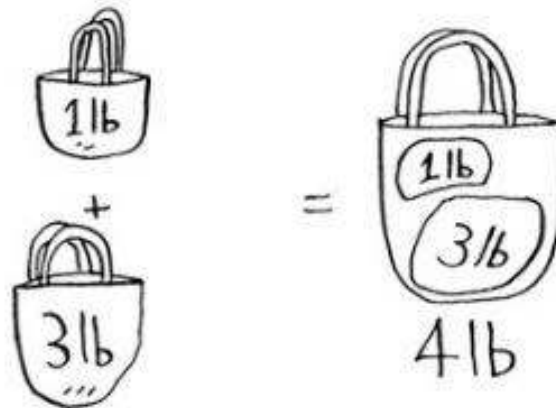
Knapsack problem: Simple solution

- ทดลองหยิบของทุกกรณีที่เป็นไปได้ทั้งหมดตามเงื่อนไข แล้วเลือกกรณีที่ให้มูลค่ารวมของที่ยหยิบมากที่สุด $\rightarrow O(2^n)$



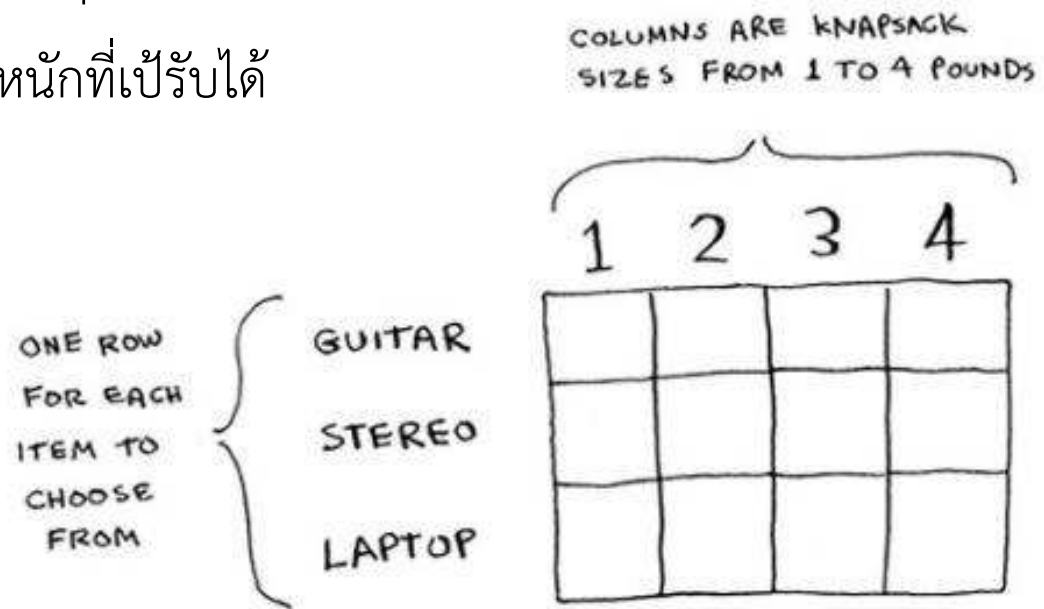
Knapsack: Dynamic Programming

- พยายามย่อยปัญหาและแก้ไขปัญหาย่อยๆ ก่อน
 - ให้คิดเหมือนเป้ของเรามีขนาดเล็กลงหลายใบ และเราพยายามใส่ของในเป้เล็กแต่ละใบให้ได้มูลค่ามากที่สุด
 - การแก้ปัญหาลใหญ่ = การนำผลลัพธ์การแก้ปัญหาย่อยมารวมกัน



Knapsack: Dynamic Programming

- แนวคิดในการทำ Dynamic Programming ง่ายๆ จะทำได้ด้วยการใช้ Grid
 - Row : สิ่งของต่างๆ
 - Column : น้ำหนักที่เป็รับได้



Knapsack: Dynamic Programming

- เริ่มเติมค่าเข้าไปใน Grid
 - เริ่มที่แถว Guitar -> คิดเหมือนว่า ในร้านมีกีตาร์ให้หยิบอย่างเดียว
 - ให้เขียนมูลค่าของทีมากที่สุดที่จะหยิบได้ไม่เกินน้ำหนักเป้ ไล่จากเป้ขนาด 1 ปอนด์ ถึง 4 ปอนด์ (เหมือนแก้ปัญหามาจากปัญหาย่อยไปใหญ่)

	1	2	3	4
GUITAR	\$1500 G	\$1500 G	\$1500 G	\$1500 G
STEREO				
LAPTOP				

← OUR CURRENT BEST GUESS FOR WHAT THE THIEF SHOULD STEAL: THE GUITAR FOR \$1500

Knapsack: Dynamic Programming

- แถว Stereo
 - คิดเหมือนว่า เราสามารถหยิบได้ทั้ง Stereo และ Guitar
 - หมายถึง เราสามารถหยิบของจากแถวด้านบนมาได้

	1	2	3	4
GUITAR	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G
STEREO	\$1500 G	\$1500 G	\$1500 G	\$3000 G
LAPTOP				

✱ เรือที่ดีที่สุด

① เลือกของเดิม (ไม่ใช่ของใหม่)

② เลือกของใหม่ (row ใหม่)

Knapsack: Dynamic Programming

- แกว Laptop
 - คิดเหมือนว่า เราสามารถหยิบได้ทั้ง Laptop, Stereo และ Guitar
 - ทำเหมือนเดิม

	1	2	3	4
GUITAR	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G
STEREO	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$3000 S
LAPTOP	\$1500 ↓ G	\$1500 ↓ G	\$2000 L	

	1	2	3	4
GUITAR	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G
STEREO	\$1500 ↓ G	\$1500 ↓ G	\$1500 ↓ G	\$3000 S
LAPTOP	\$1500 ↓ G	\$1500 ↓ G	\$2000 L	\$3500 L G

↑
THE ANSWER!

$$\$3000 \text{ STEREO} \text{ VS } \left(\$2000 \text{ LAPTOP} + \frac{???}{1 \text{ LB OF FREE SPACE}} \right)$$

Knapsack: Dynamic Programming

- คำตอบสุดท้าย -> Guitar + Laptop -> \$3500
- ถ้าสังเกตดีๆ จะเห็นว่ามีการคิดซ้ำๆ และสามารถเขียนสูตรในรูปแบบของ Recursion เพื่อเก็บค่าใน Grid ได้ดังนี้

↓ ROW ↓ COLUMN

$$\text{CELL}[i][j] = \max \left\{ \begin{array}{l} 1. \text{ THE PREVIOUS MAX (VALUE AT CELL } [i-1][j]) \\ \text{VS} \\ 2. \text{ VALUE OF CURRENT ITEM + VALUE OF THE REMAINING SPACE} \\ \quad \quad \quad \uparrow \\ \quad \quad \quad \text{CELL}[i-1][j - \text{ITEM'S WEIGHT}] \end{array} \right.$$

↑
จากค่าเดิม / ว่างเดิม

บวก

น้ำหนักของ

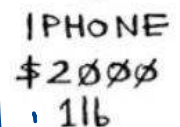
- สมมุติว่ามีของในร้านเพิ่มอีก 1 อย่าง คือ iPhone
- ให้แก้ปัญห Knapsack ของเป้ขนาด 4 ปอนด์ -> จะต้องหยิบ
สิ่งของใดบ้าง ที่ทำให้ได้มูลค่าของรวมกันมากที่สุด

২৫৭

7. សង្គម

เขียนขึ้นใหม่

↑
NEW ANSWER



อิน → ก่อ 79 น
น → อิน → 79 น
อิน 79 น

แนวคิด Dynamic Programming

- สร้าง Grid เพื่อใช้ในการเก็บผลลัพธ์ย่อยๆ และนำไปแก้ปัญหาลใหญ่ได้เร็วขึ้น (ไม่ต้องคำนวณใหม่)
- ใส่ค่าเข้าไปในแต่ละเซลล์ใน Grid โดยค่าที่ใส่นั้นมักจะเป็นค่าที่สะท้อนถึงความคุ้มค่า หรือทางที่ดีที่สุดของการแก้ปัญห
 - ค่าเซลล์ใน Knapsack คือ มูลค่าสินค้าที่หยิบ
- แต่ละเซลล์ใน Grid จะหมายถึงคำตอบของปัญหาย่อย

Longest Common Substring

- ปัญหา : ให้หาระดับความคล้ายคลึงระหว่างคำ 2 คำ
 - ใช้ในกรณีค้นหาคำศัพท์ แต่อาจจะพิมพ์ผิด โปรแกรมสามารถเดาคำศัพท์ที่มีความคล้ายคลึงกับคำที่หาให้ได้
 - HISH & FISH
 - HISH & VISTA
 - สูตรการคำนวณควรเป็นอย่างไร

	H	I	S	H
F	0	0		
I				
S			2	0
H				3

Longest Common Substring

```
if word_a[i] == word_b[j] :  
    cell[i][j] = cell[i-1][j-1] + 1  
else:  
    cell[i][j] = 0
```

1. IF THE LETTERS
DONT MATCH,
THE VALUE IS
ZERO.

	H	I	S	H
F	0	0	0	0
I	0	1	0	0
S	0	0	2	0
H	0	0	0	3

2. IF THEY DO MATCH,
THIS VALUE IS
VALUE OF TOP-LEFT NEIGHBOR + 1

Longest Common Substring

- HISH & VISTA
- คำตอบไม่จำเป็นต้องเป็นเซลล์สุดท้าย
- กรณีนี้ คำตอบคือค่าที่มากที่สุดใน Grid
- HISH และ VISTA มีค่าความคล้ายเป็น 2

	V	I	S	T	A
H	0	0	0	0	0
I	0	1	0	0	0
S	0	0	2	0	0
H	0	0	0	0	0

FINAL ANSWER NOT THE FINAL ANSWER

Longest Common Subsequence

- วิธีหาความคล้ายของคำก่อนหน้านี้ อาจจะทำให้ผลลัพธ์ที่ไม่ดีในบางกรณี เช่น
 - ความคล้ายของ FOSH และ FORT เป็น 2
 - ความคล้ายของ FOSH และ FISH เป็น 2 ☹️

	F	O	S	H
F	1	0	0	0
O	0	2	0	0
R	0	0	0	0
T	0	0	0	0

vs

	F	O	S	H
F	1	0	0	0
I	0	0	0	0
S	0	0	1	0
H	0	0	0	2

Longest Common Subsequence

- ลองเพิ่มเติมวิธีคิด
 - แทนที่จะหาขนาดข้อความย่อย (Substring) ที่เหมือนกัน
 - พิจารณา Subsequence ที่เหมือนกันมากที่สุดแทน
 - Subsequence ของ FOSH คือ $\langle F, O \rangle$, $\langle F, S \rangle$, $\langle F, H \rangle$, $\langle F, O, S \rangle$, $\langle F, O, H \rangle$, ...

	F	O	S	H
F	1	1		
I	1			
S		1	2	2
H				

Longest Common Subsequence

```
if word_a[i] == word_b[j] :
```

```
cell[i][j] = cell[i-1][j-1] + 1
```

else:

```
cell[i][j] = max(cell[i-1][j], cell[i][j-1])
```

7. $\mu_{\text{H}_2\text{O}} = 0.018$

ഒരു വാക്ക്

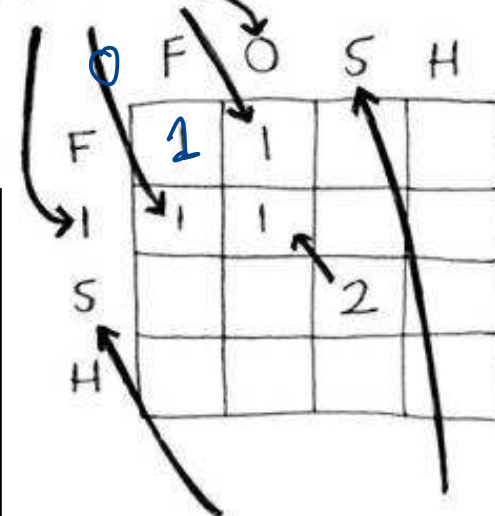
တာဝန်ယူ

រ៉ាប់រង ចាត់រៀន

OF THE TOP AND
LEFT NEIGHBORS

1. IF THE LETTERS
DONT MATCH,
PICK THE LARGER

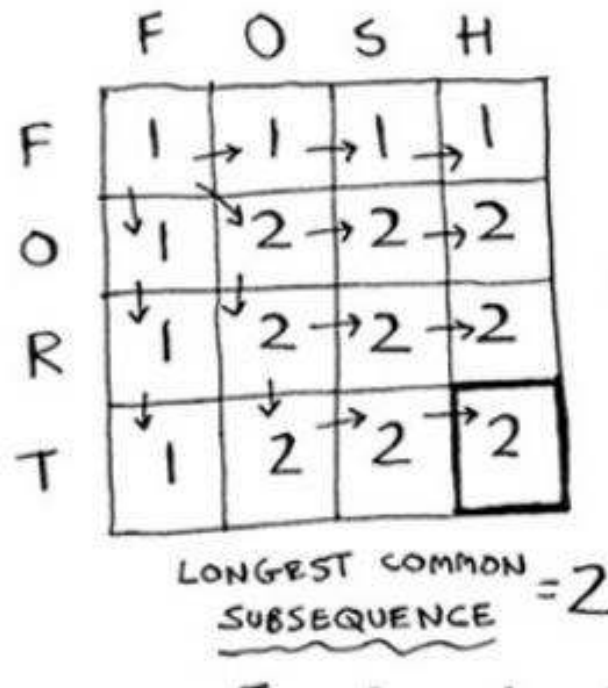
(DIFFERENT FROM
LONGEST COMMON
SUBSTRING)



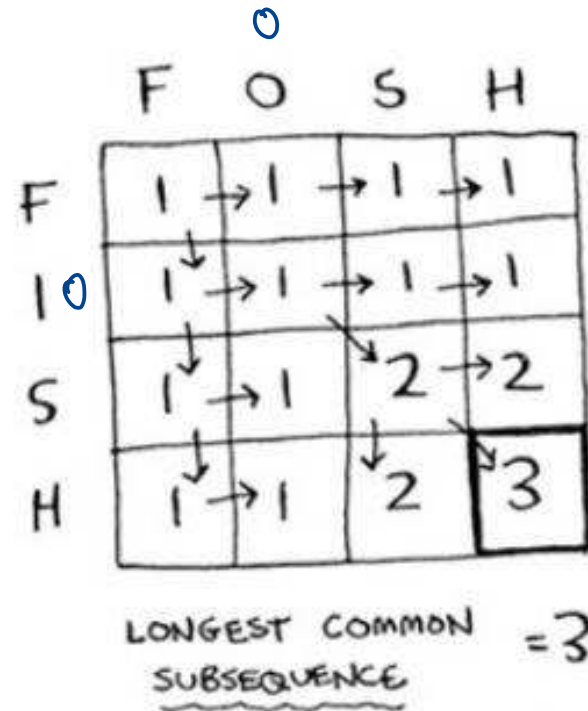
2. IF THEY DO MATCH,
THIS VALUE IS
VALUE OF TOP-LEFT NEIGHBOR + 1
(JUST LIKE LONGEST COMMON SUBSTRING)

Longest Common Subsequence

6th Aug 2017



vs



Quiz

- สมมุติว่าคุณเดินทางไปประเทศอังกฤษ และมีเวลาท่องเที่ยวเป็นเวลา 2 วัน คุณควรวางแผนจะไปสถานที่ท่องเที่ยวต่างๆ อย่างไร ที่จะได้ผลรวมเรตติ้งของสถานที่ที่ไปมากที่สุด
 - ให้เขียนสูตรที่ใช้ในการทำ Dynamic Programming (Recursion)
 - ให้เขียนและเติมค่า Grid ที่ใช้ในการคำนวณ Dynamic Programming

ATTRACTION	TIME	RATING
WESTMINSTER ABBEY	1/2 DAY	7
GLOBE THEATER	1/2 DAY	6
NATIONAL GALLERY	1 DAY	9
BRITISH MUSEUM	2 DAYS	9
ST. PAUL'S CATHEDRAL	1/2 DAY	8

Quiz

- จงหาค่า Longest Common Substring ของคำว่า “together” และ “tighter” (พร้อมแสดง Grid และค่าใน Grid)
- จงหาค่า Longest Common Subsequence ของคำว่า “together” และ “tighter” (พร้อมแสดง Grid และค่าใน Grid)